

CMPS 405: OPERATING SYSTEMS

Memory Management: Main Memory

This material is based on the operating system books by Silberschatz and Stallings and other Linux and system programming books.

Objectives

- ❖ To introduce the terminology of **main memory**.
- ❖ To present a number of different methods for **memory allocation** and their relevant **problems** and available **solutions**.

Topics

- ❖ Motivation
- ❖ Binding of Instructions and Data to Memory
 - Logical vs. Physical Address Space
 - Overlay
 - Swapping
- ❖ Contiguous Memory Allocation
 - Dynamic Storage-Allocation Problem
 - Internal and External Fragmentation
- ❖ Non-Contiguous Memory Allocation
 - Paging
 - Segmentation

RAM (Random-access Memory)

- ❖ **RAM** is volatile in the sense that it loses its data if it is powered off.
- ❖ **RAM** comes in two varieties:
 - **SRAM** (Static RAM)
 - **DRAM** (Dynamic RAM)

RAM (Random-access Memory)

❖ **SRAM** (Static RAM)

- Faster and significantly more expensive than DRAM.
- Used for cache memories, both on and off the CPU chip.
- A computer system has few megabytes of SRAM.
- The cell can stay indefinitely in either of two voltage configurations, states, as long as it is kept powered.

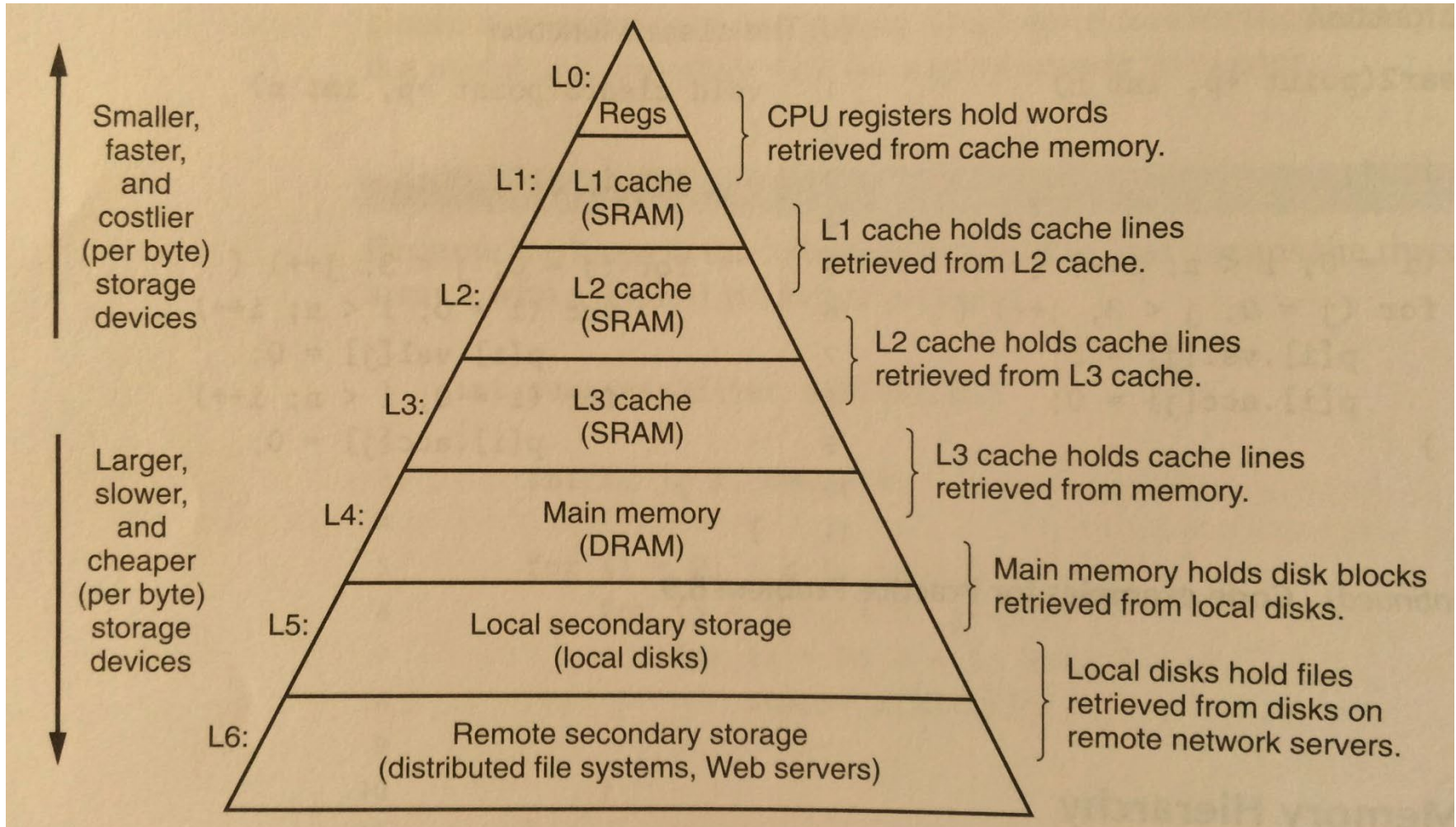
❖ **DRAM** (Dynamic RAM)

- Slower and much cheaper than SRAM
- Used for the main memory and frame buffer of a graphics system.
- A computer system has thousands of megabytes of DRAM.
- It requires periodic refreshing, which adds latency.

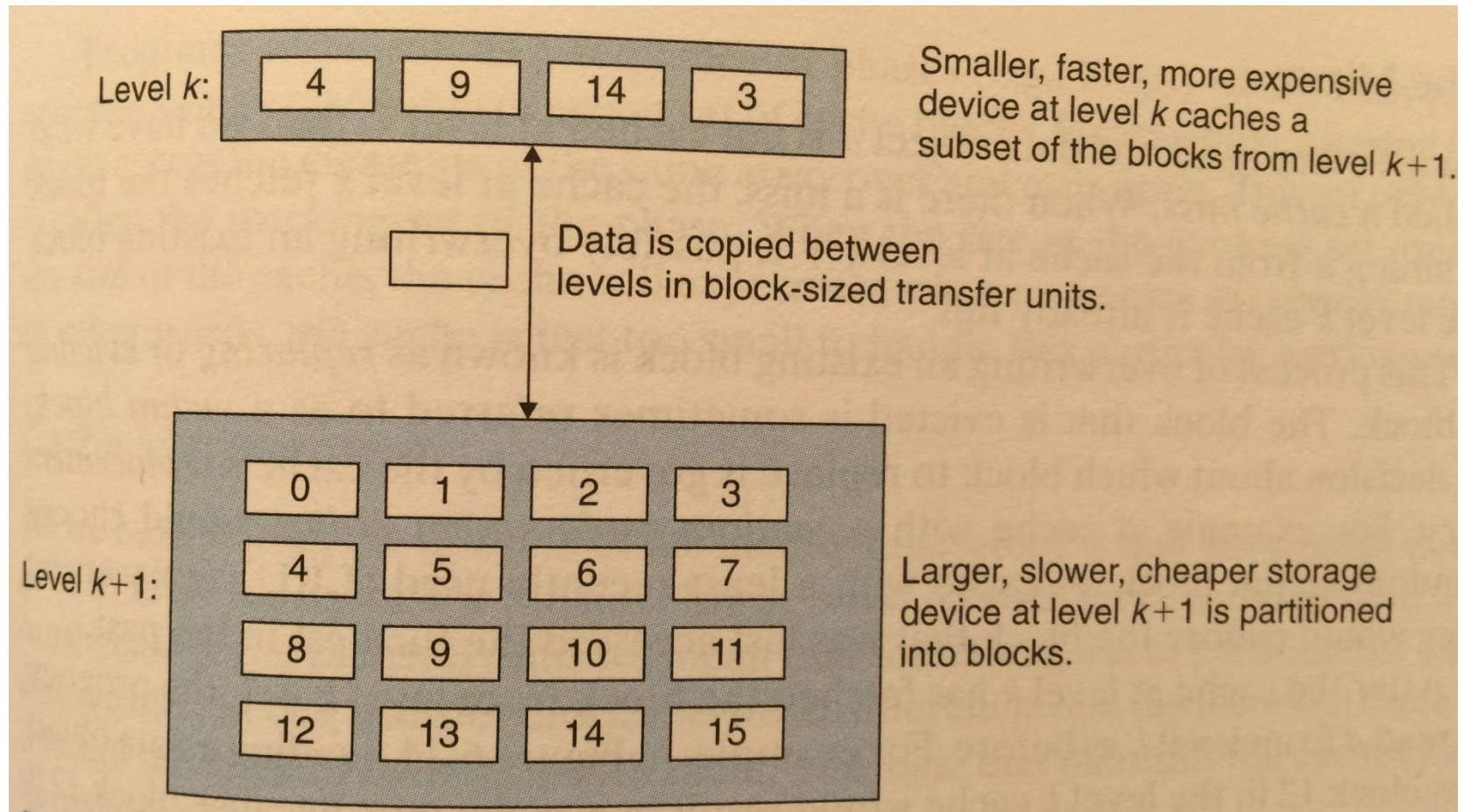
ROM (Read-only Memory)

- ❖ Historically called so even though they can be written.
- ❖ **ROMs** are nonvolatile memories, retaining their information even when they are powered off.
- ❖ **There are varieties of ROMs:**
 - **Programmable ROM (PROM):** can be programmed exactly once.
 - **Erasable programmable ROM (EPROM):** can be programmed using a special device to write on EPROM. They can be erased and programmed in the order of *1000* times.
 - **Electrical erasable PROM (EEPROM):** does not require a special device instead can be programmed and erased in-place on printed circuit cards. Can be programmed in the order of 10^5 times.
 - **Flash memory:** based on EEPROM. The solid state disk (**SSD**) is a newer form of flash-based disk.
- ❖ Programs stored in **ROM** devices are often referred to as *firmware*.

Memory Hierarchies



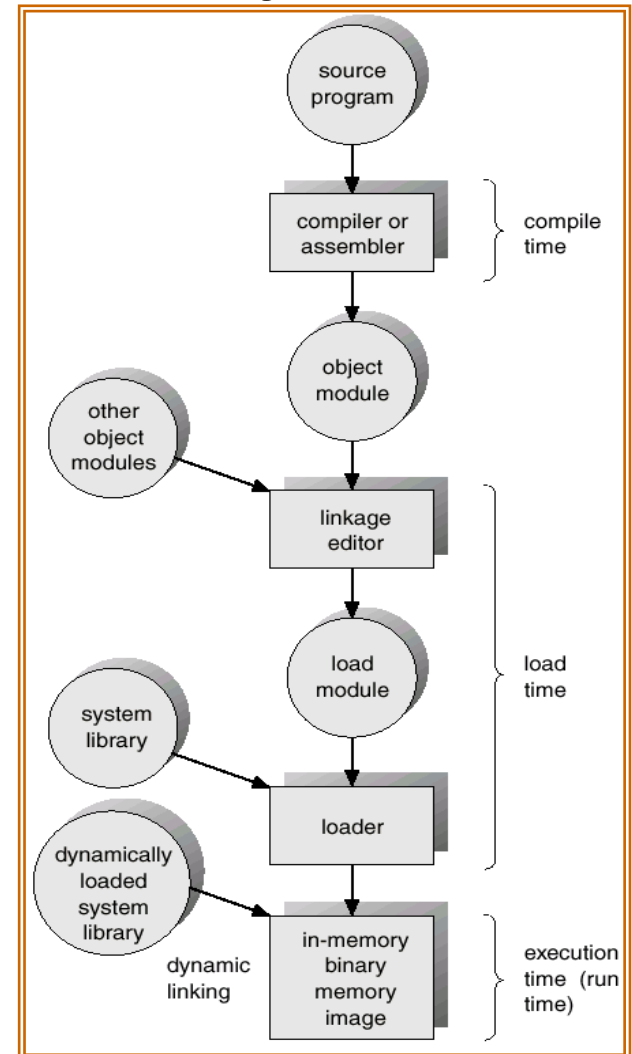
Caching Concept



Binding of Instructions and Data to Memory

Address binding of instructions and data to memory addresses can happen at three different stages

- ❖ **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
- ❖ **Load time:** Must generate **relocatable code** if memory location is not known at compile time
- ❖ **Execution time:** Binding delayed until **run time** if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., *base* and *limit registers*).



Logical vs. Physical Address Space

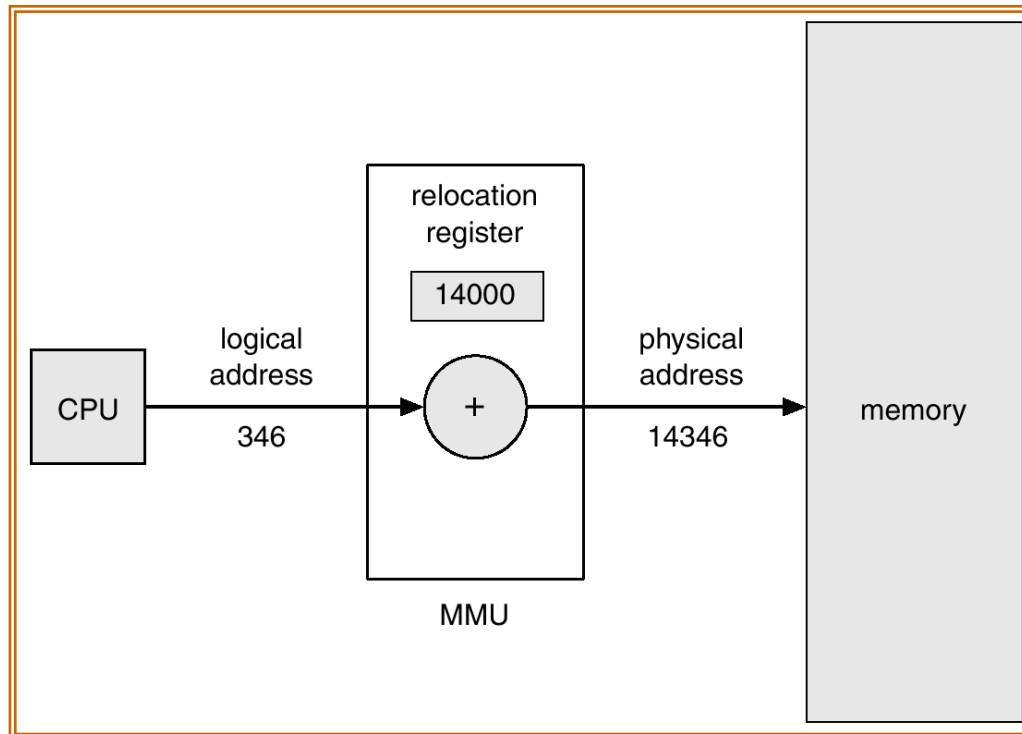
- ❖ The concept of a *logical address space* that is bound to a separate *physical address space* is central to proper **memory management**
 - **Logical address** – generated by the CPU during program execution; also referred to as *virtual address*
 - **Physical address** – actual location as seen by the physical memory unit
- ❖ **Logical to Physical Address Translation** is performed by the **Memory Management Unit (MMU)**. This unit is responsible for **mapping logical addresses** to corresponding **physical addresses** based on the **memory mapping** established by the operating system.

Memory-Management Unit (MMU)

- ❖ **MMU** is a **computer hardware component** that is usually integrated into the **processor**, although in some systems it occupies a **separate IC** (integrated circuit) chip.
- ❖ **MMU** is **responsible** for all **aspects** of **memory management**.
- ❖ A **processor with an MMU** that provides **virtual memory** has an **on-chip cache** of "**translations**". Each "**translation record entry**" tells the CPU the **mapping** of **one virtual address** to **one physical address**.

Dynamic relocation using a relocation register

- ❖ In this scheme, the **value** in the **relocation register** is **added** to **every address generated** by a **user process** at the time it is sent to memory.



Dynamic Loading

❖ Routine is not loaded until it is called

- Useful when large amounts of code are needed to handle infrequently occurring cases.
- Better memory-space utilization.
- The main program is loaded into memory and executed.
- A routine is loaded only when it is needed (called).
- Unused routine is never loaded.
- No special support from the operating system is required. It is the responsibility of the programmers to design their programs to take advantage of such a method. **Operating systems** may help the programmer, however, by providing **library routines** to **implement dynamic loading**.

Overlays

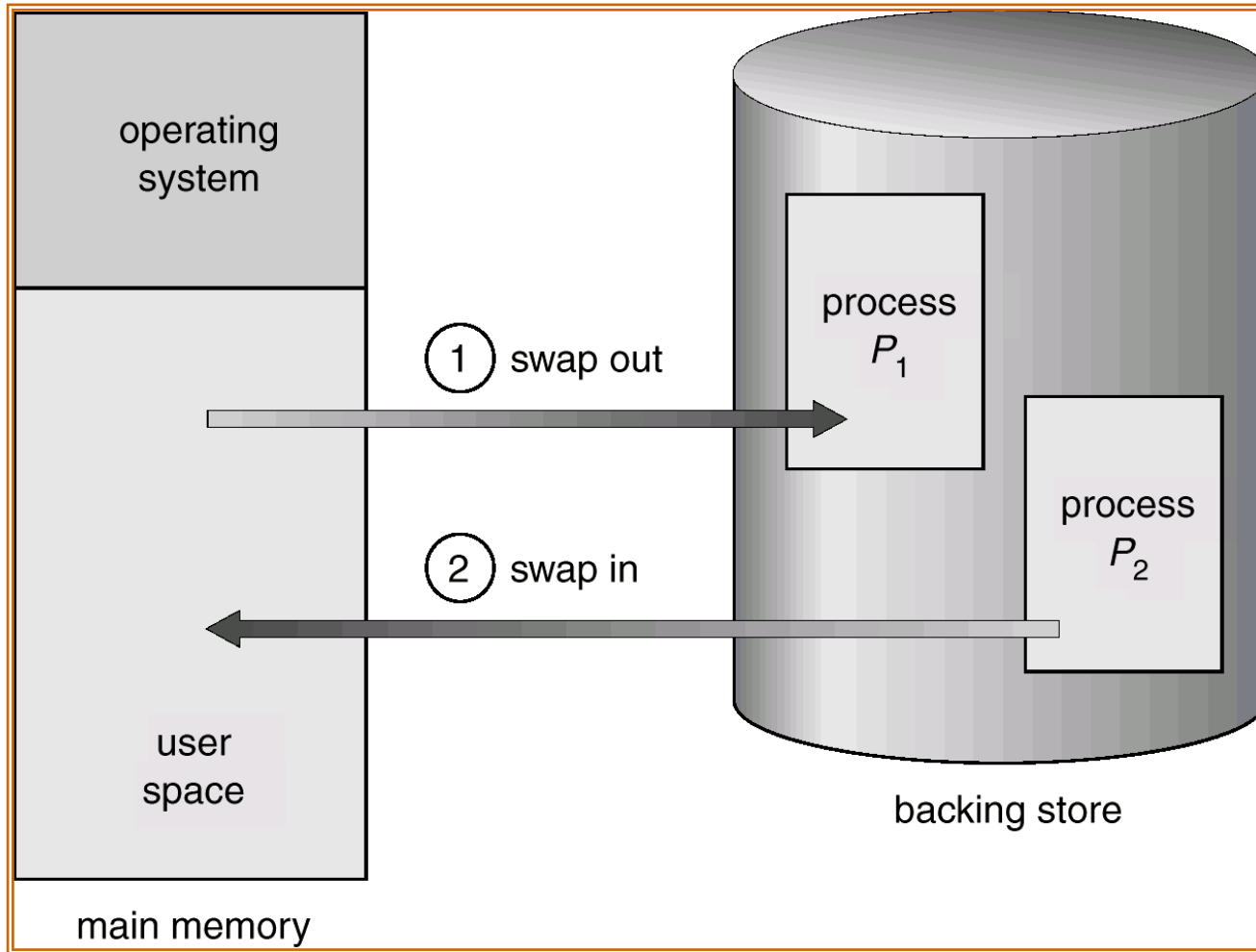
- ❖ In memory management, **overlays** refer to a technique used to overcome the limitations of memory constraints, particularly in systems with **limited physical memory**.
- ❖ Needed when **process** (program) is **larger** than amount of **physical memory allocated** to it.
- ❖ The idea behind **overlays** is to **Manually** divide a program into pieces (“**overlays**”) that share the same memory region but are loaded at different times.
- ❖ **Only necessary overlays of a program are loaded into memory at a given time**, freeing up memory for other tasks. The **unused overlays of the program are kept on disk or other storage**, and are loaded into memory as needed.
- ❖ As the program execution progresses, when a **program overlay is no longer needed** and **another is required**, the operating system **switches** the relevant overlays in and out of memory. This is done dynamically during program execution.
- ❖ **Implemented by user**, no special support needed from operating system, programming design of overlay structure is complex.



Swapping

- ❖ A **process** can be **swapped temporarily out of memory to a backing store**, and **then brought back into memory for continued execution**
- ❖ **Backing store** – **fast disk large enough** to accommodate copies of all **memory images** for all users; must provide direct access to these memory images
- ❖ **Roll out, roll in** – **swapping variant** used for **priority-based scheduling algorithms**; lower-priority process is swapped out so higher-priority process can be loaded and executed
- ❖ Major part of **swap time** is transfer time; total transfer time is directly proportional to the amount of memory swapped
- ❖ Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)

Schematic View of Swapping



Contiguous Allocation

- ❖ **Main memory** is usually divided into only **two partitions**:
 - **Resident operating system**, usually held in **low memory** with interrupt vector.
 - **User processes** then held in **high memory**.

Contiguous Allocation

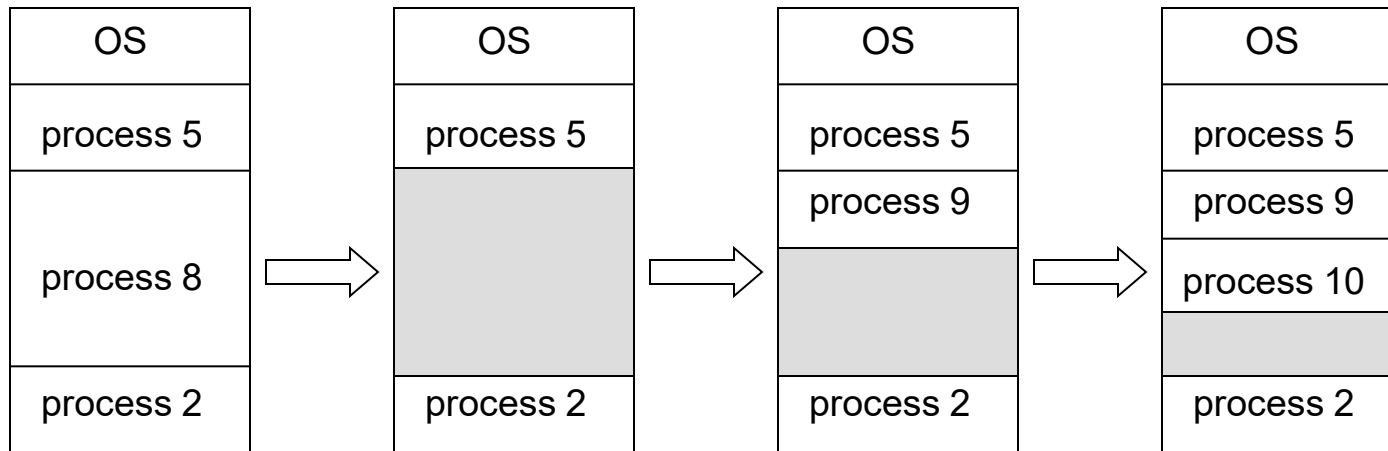
❖ Single-partition allocation

- **Main memory** is usually divided into only **two partitions**:
 - One single partition for operating system.
 - One single partition for ONE User process.
- So only one program can run in memory at a time.

Contiguous Allocation

❖ Multiple-partition allocation

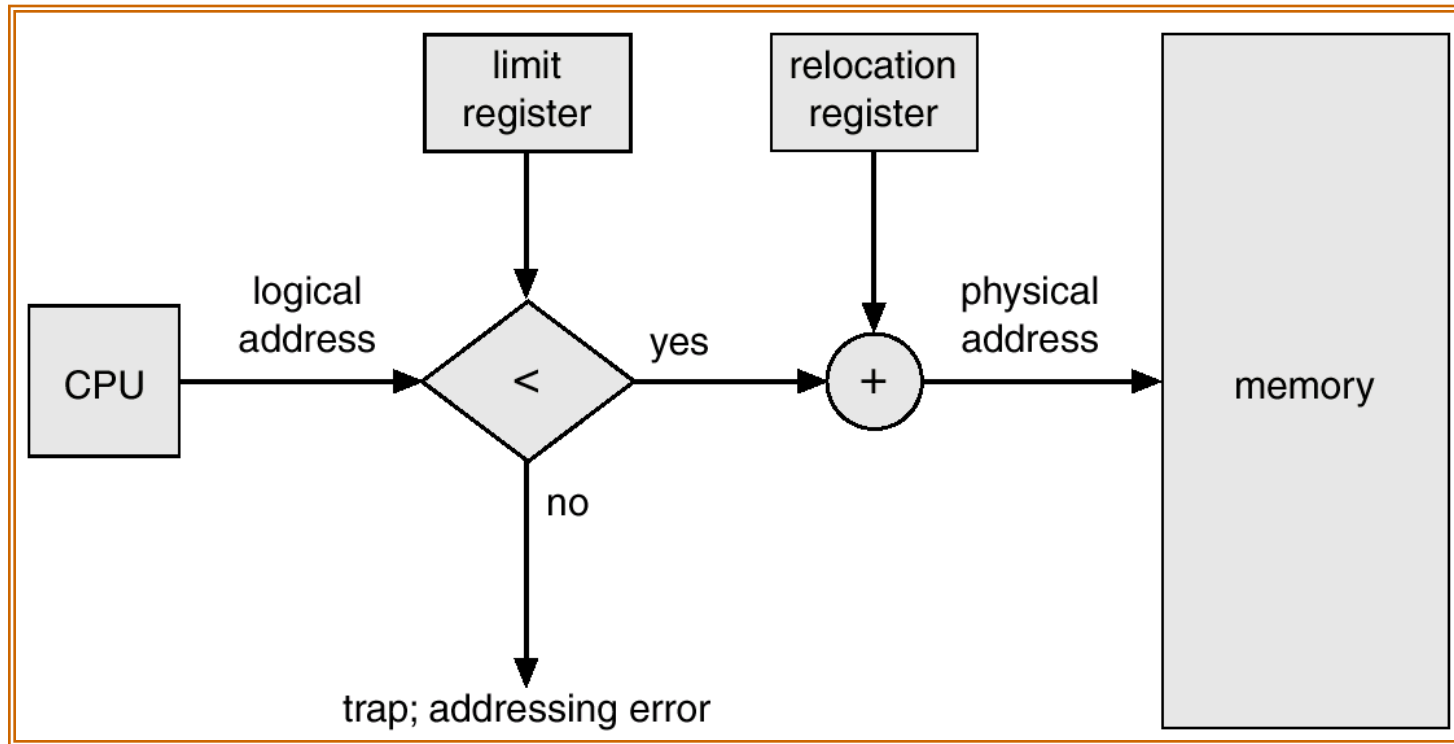
- **Hole** – block of available memory; holes of various size are scattered throughout memory
- When a process arrives, it is allocated memory from a **hole** large enough to accommodate it
- Operating system maintains information about:
a) allocated partitions b) free partitions (hole)



Contiguous Allocation

- ❖ **Relocation-register scheme** used to **protect user processes** from each other, and from changing operating-system code and data
 - **Relocation register** contains value of smallest physical address
 - **Limit register** contains range of logical addresses – each logical address must be less than the limit register.

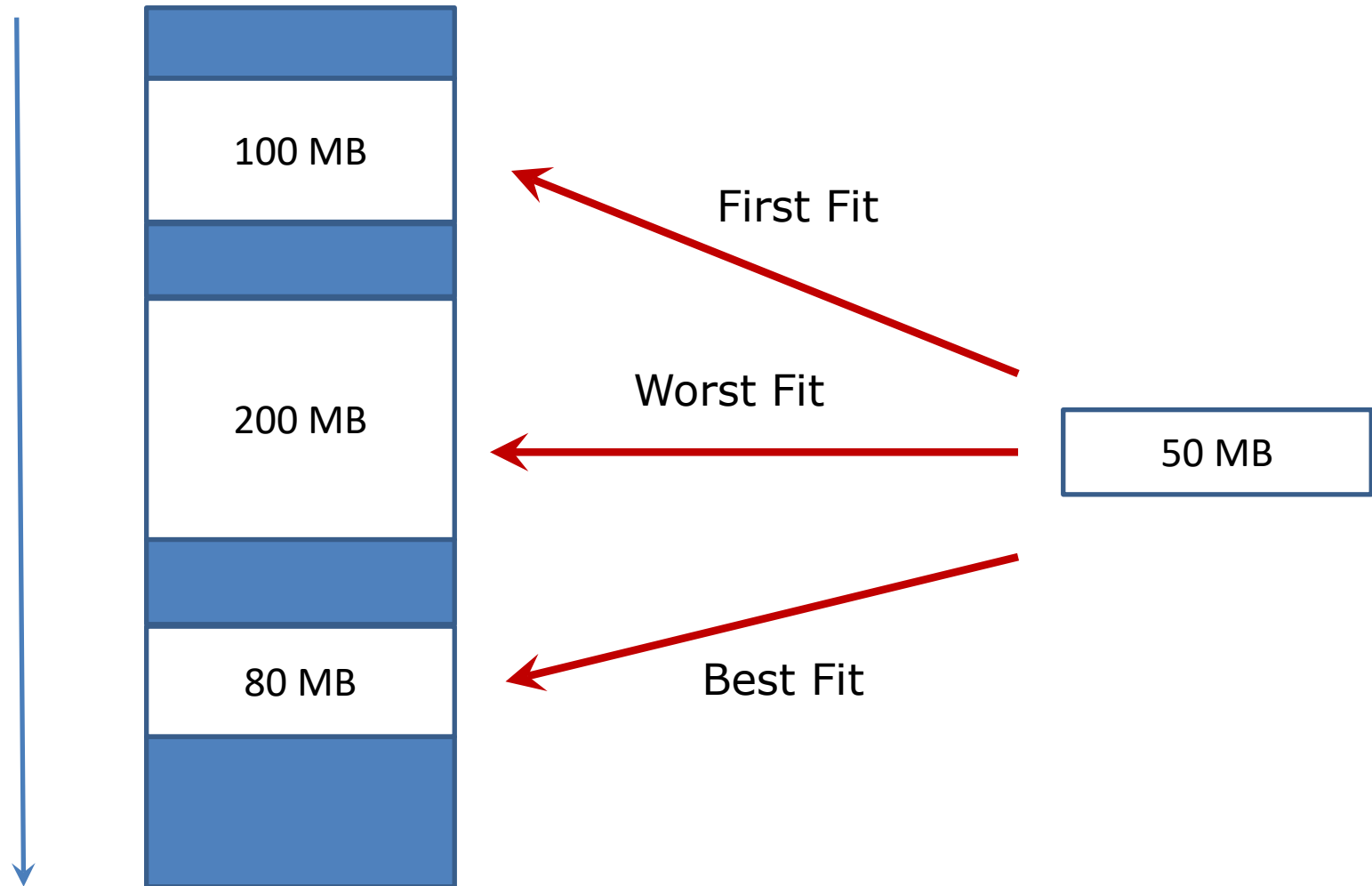
Hardware Support for Relocation and Limit Registers



Dynamic Storage-Allocation Problem

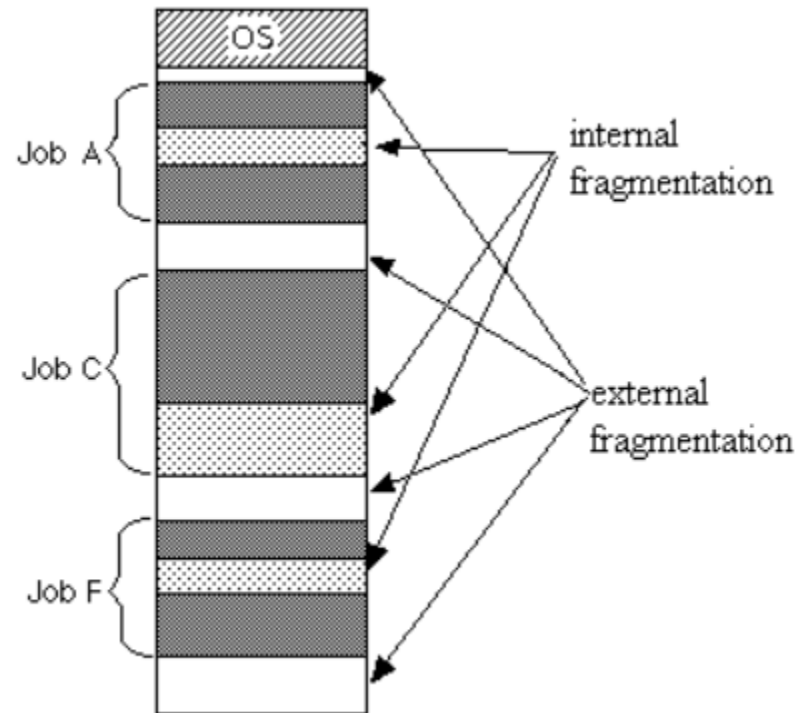
How to satisfy a **request** of **size n** from a **list of free holes**

- ❖ **First-fit**: Allocate the ***first* hole** that is big enough
- ❖ **Best-fit**: Allocate the ***smallest* hole** that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole.
- ❖ **Worst-fit**: Allocate the ***largest* hole**; must also search entire list. Produces the largest leftover hole.



Fragmentation

- ❖ **External Fragmentation** – total memory space exists to satisfy a request, but it is **not contiguous**.
- ❖ **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.

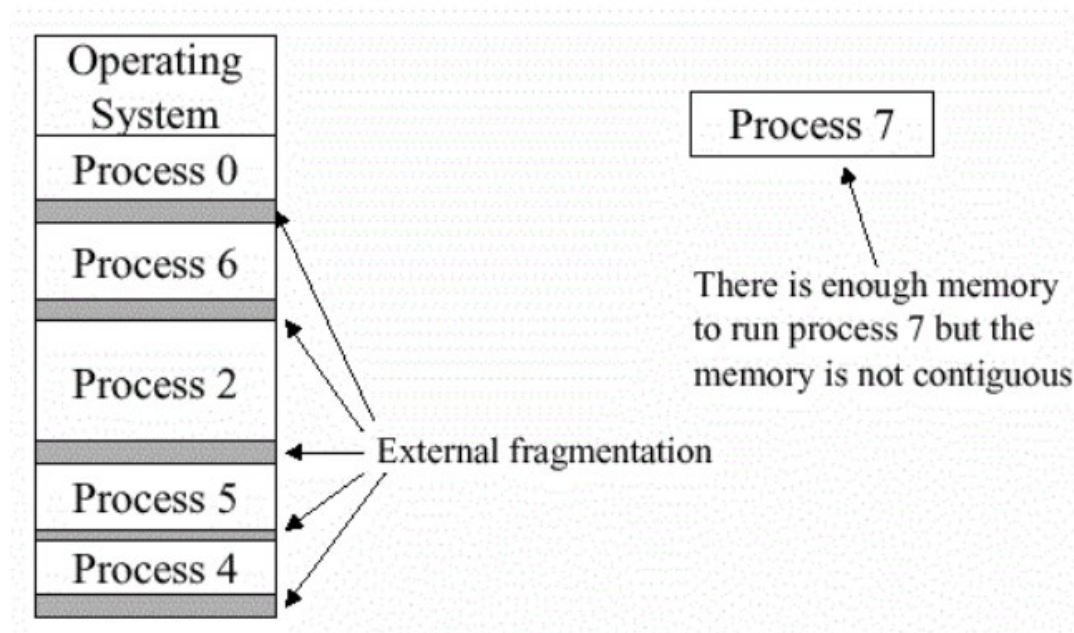


<https://stackoverflow.com/questions/1200694/internal-and-external-fragmentation>

Fragmentation

❖ Reduce external fragmentation by **compaction**

- Shuffle memory contents to place all free memory together in **one large block**.
- Compaction is **possible *only*** if relocation is **dynamic**, and is done at **execution time**.



<https://stackoverflow.com/questions/1200694/internal-and-external-fragmentation>

Motivation to non-contiguous memory allocation

- ❖ The aim of this section is to present **memory-management schemes** allowing the **physical address space of a process** to be **non-contiguous**.
- ❖ This aim is **achieved** by **memory allocation** schemes such as **paging** and **segmentation**.
- ❖ Both schemes **require hardware support**.
- ❖ **Paging** and **Segmentation** can also be used **together** forming another memory allocation scheme.

Paging

How does it work?

❖ Pre-memory allocation:

- Divide **physical memory** into fixed-sized blocks called **frames** (size is power of 2, usually between 512 bytes and 8,192 bytes depending on the computer architecture)
- Divide **logical memory** into blocks of same size called **pages**
- Keep track of all free frames

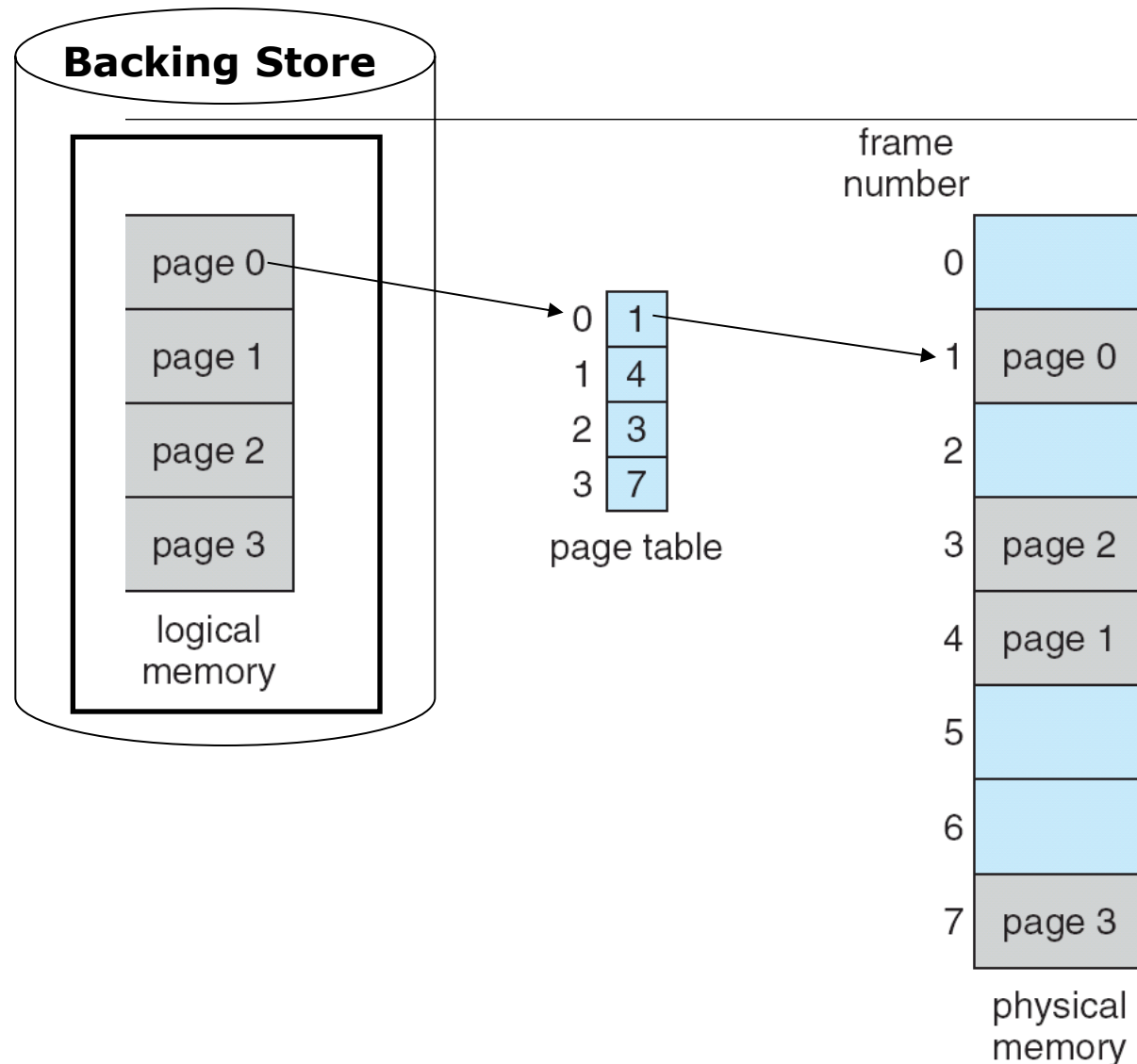
❖ Memory allocation:

- To run a program of size **n pages**, need to find **n free frames** and load program from backing store to main memory.
- Set up a **page table** to **translate logical to physical addresses**

****Some CPUs and Kernels support:**

- **Large page sizes say 16 MB or**
 - **Multiple page sizes: For example Solaris uses page sizes of 8K and 4 MB. Or**
 - **Variable on-the-fly page size.**

Paging Model of Logical and Physical Memory



+S VS. -S

❖ Drawback

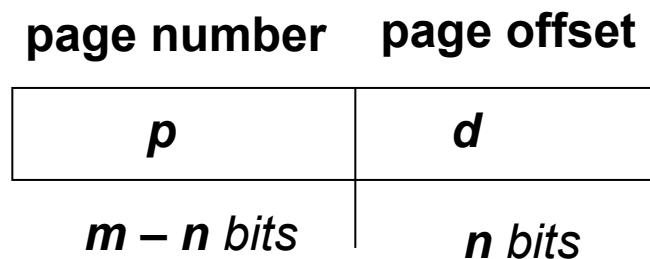
- **Internal fragmentation:** the last frame may not be completely full.
 - **Example:** if **page size** is **2,048 bytes**, a **process** of **72,766 bytes** would need **35 pages** plus **1,086 bytes**. The process is **allocated 36 frames** in which frame number **36** has **internal fragmentation** of **962 bytes** ($2048 - 1086$).
 - What is the worst case of such internal fragmentation?
 - What is the average case of such internal fragmentation?
- **Separates** between the **user's logical view** of memory and the **actual physical memory**.

❖ Strength

- **Paging** solves the **problem of storing memory blocks of different sizes on the backing store**. Without paging, this would lead to **fragmentation**, and unlike main memory, **the backing store cannot be compacted efficiently** because it is much slower to access.
- **No external fragmentation.**

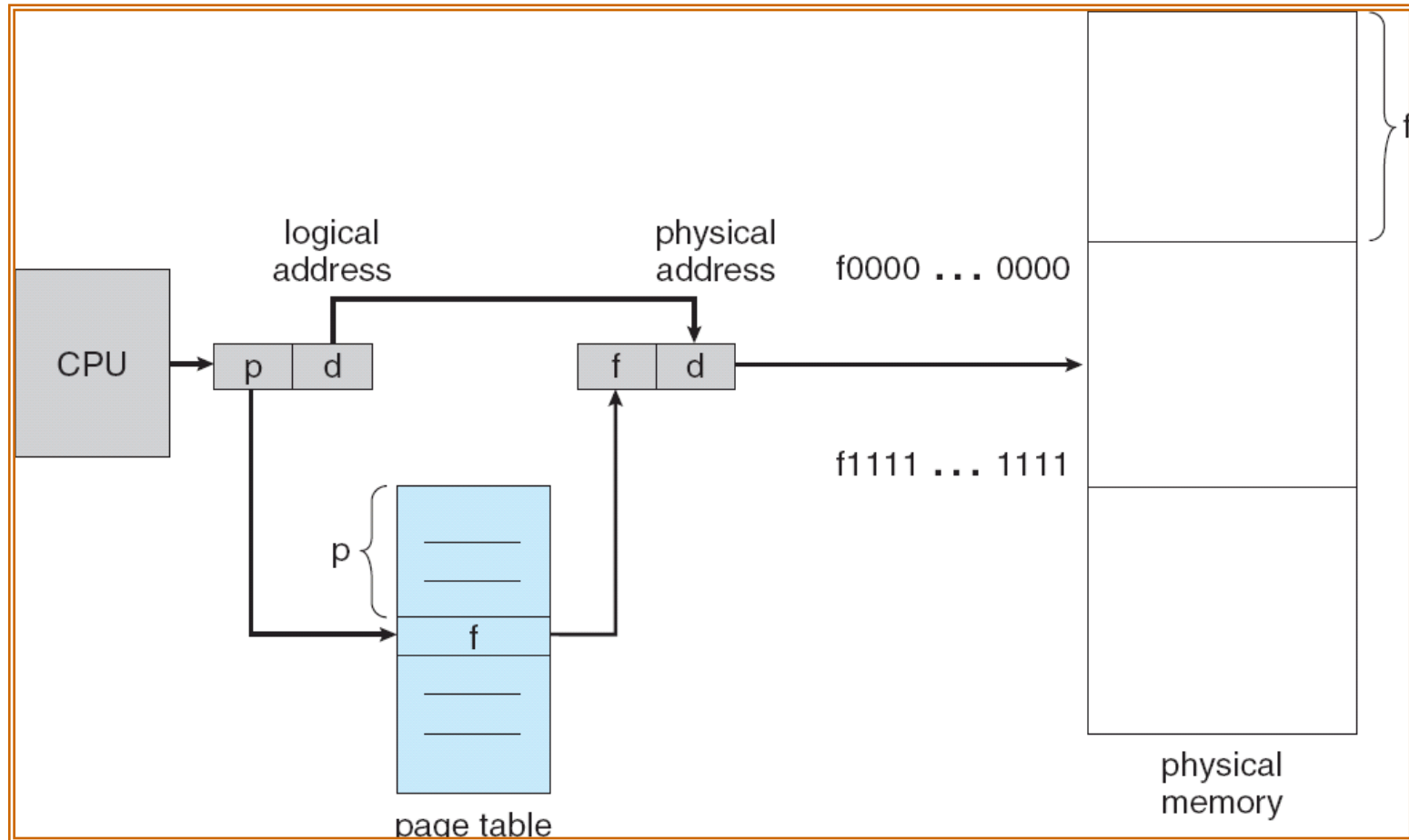
Address Translation Scheme

- ❖ Address generated by CPU (**virtual** or **logical address**) is divided into two parts:
 - **Page number (p)** – used as an index into a *page table* which contains base address of each page in physical memory.
 - **Page offset (d)** – to be combined with base address to define the physical memory address.



A virtual address for given logical address space 2^m , page size 2^n , and total number of pages 2^{m-n} .

Paging Hardware



Practice

You will need this table for fast lookup in solving the practical exercises.

Powers of 2	Decimal Byte	Memory Unit	Powers of 2	Decimal Byte	Memory Unit	Powers of 2	Decimal Byte	Memory Unit
2^0	1	1 B	2^{12}	4,096	4 KB	2^{24}	16,777,216	16 MB
2^1	2	2 B	2^{13}	8,192	8 KB	2^{25}	33,554,432	32 MB
2^2	4	4 B	2^{14}	16,384	16 KB	2^{26}	67,108,864	64 MB
2^3	8	8 B	2^{15}	32,768	32 KB	2^{27}	134,217,728	128 MB
2^4	16	16 B	2^{16}	65,536	64 KB	2^{28}	268,435,456	256 MB
2^5	32	32 B	2^{17}	131,072	128 KB	2^{29}	536,870,912	512 MB
2^6	64	64 B	2^{18}	262,144	256 KB	2^{30}	1,073,741,824	1 GB= 1024^3
2^7	128	128 B	2^{19}	524,288	512 KB	2^{31}	2,147,483,648	2 GB
2^8	256	256 B	2^{20}	1,048,576	1 MB= 1024^2	2^{32}	4,294,967,296	4 GB
2^9	512	512 B	2^{21}	2,097,152	2 MB	2^{33}	8,589,934,592	8 GB
2^{10}	1024	1 KB	2^{22}	4,194,304	4 MB	2^{34}	17,179,869,184	16 GB
2^{11}	2,048	2 KB	2^{23}	8,388,608	8 MB	2^{35}	34,359,738,368	32 GB

**Note that page numbering starts from 0.*

Notes on Calculation of logical and physical addresses

- ❖ Assume that a **logical address LA** is mapped to **physical address PA**, in a page system that has page size of **K** bytes then:
 - $Page\ number = \left\lfloor \frac{LA}{K} \right\rfloor$ (floor operation)
 - $Frame\ number = \left\lfloor \frac{PA}{K} \right\rfloor$
 - $Offset = mod\left(\frac{LA}{K}\right) = mod\left(\frac{PA}{K}\right)$
- ❖ **Example:** Given a paging system with **page size 2KB**. Assume that a **logical address 20,230** is mapped to a **physical address 411,398**. Find the **logical page number**, and its corresponding **physical frame number**. Also, find the value of **offset**.
 - $K = 2KB, LA = 20,230, PA = 411,398$
 - $Page\ number = \left\lfloor \frac{20230}{2048} \right\rfloor = 9$
 - $Frame\ number = \left\lfloor \frac{411398}{2048} \right\rfloor = 200$
 - $Offset = mod\left(\frac{20230}{2048}\right) = mod\left(\frac{411398}{2048}\right) = 1798$

Notes on Calculation of logical and physical addresses

- ❖ Assume that a logical address **page number P** and **offset D**, is mapped to a physical address **frame number F** and **offset D**. If the **page size is K bytes**, then:
 - *Logical address* = $P \times K + D$
 - *Physical address* = $F \times K + D$
- ❖ **Example:** Given a paging system with **page size 2KB**. Assume that a logical address with **page number 300** and **offset 234**, is mapped to **physical address** with **frame number 400**. Calculate the **logical address** and corresponding **physical address**.
 - $K = 2\text{KB}, P = 300, F = 400, D = 234.$
 - *Logical address* = $300 \times 2048 + 234 = 614,634$
 - *Physical address* = $400 \times 2048 + 234 = 819,434$

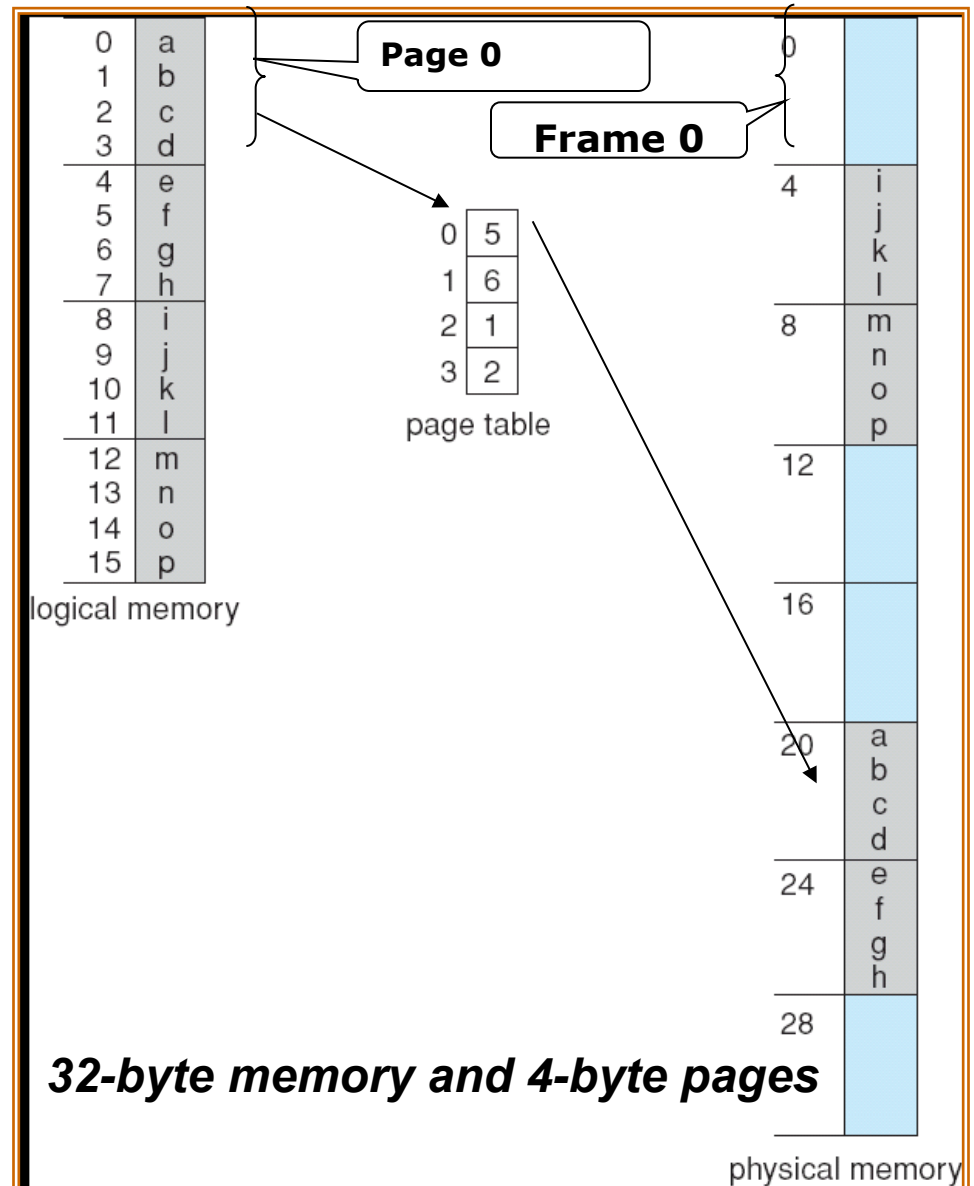
Paging Example

Logical address 0:
page 0 offset 0.
Mapped to physical address:
 $(5 \times 4 + 0) = 20$

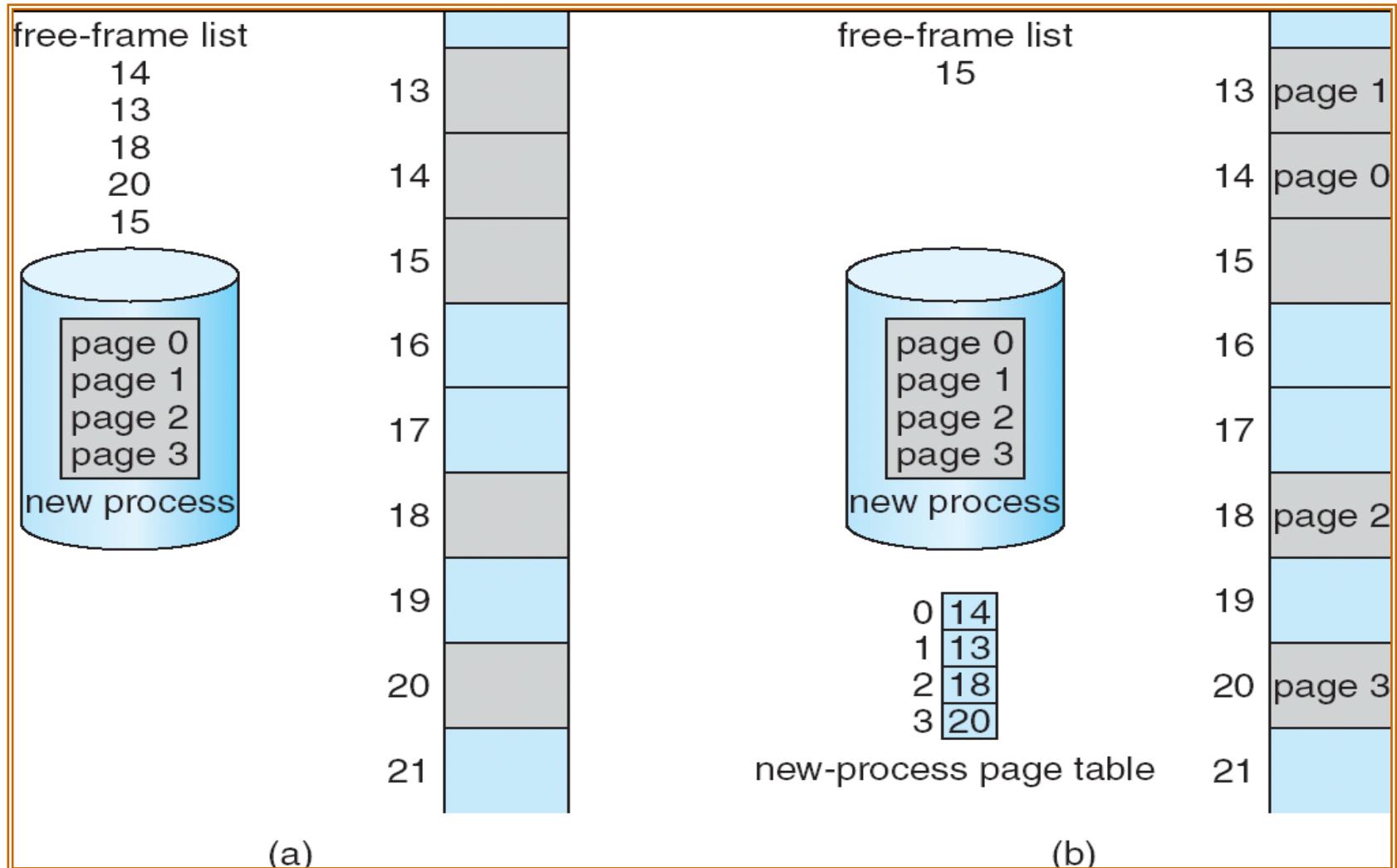
Logical address 3:
page 0 offset 3.
Mapped to physical address:
 $(5 \times 4 + 3) = 23$

Logical address 4:
page 1 offset 0.
Mapped to physical address:
 $(6 \times 4 + 0) = 24$

Logical address 13:
page 3 offset 1.
Mapped to physical address:
 $(2 \times 4 + 1) = 9$



Free Frames



Before allocation

After allocation

Shared Pages

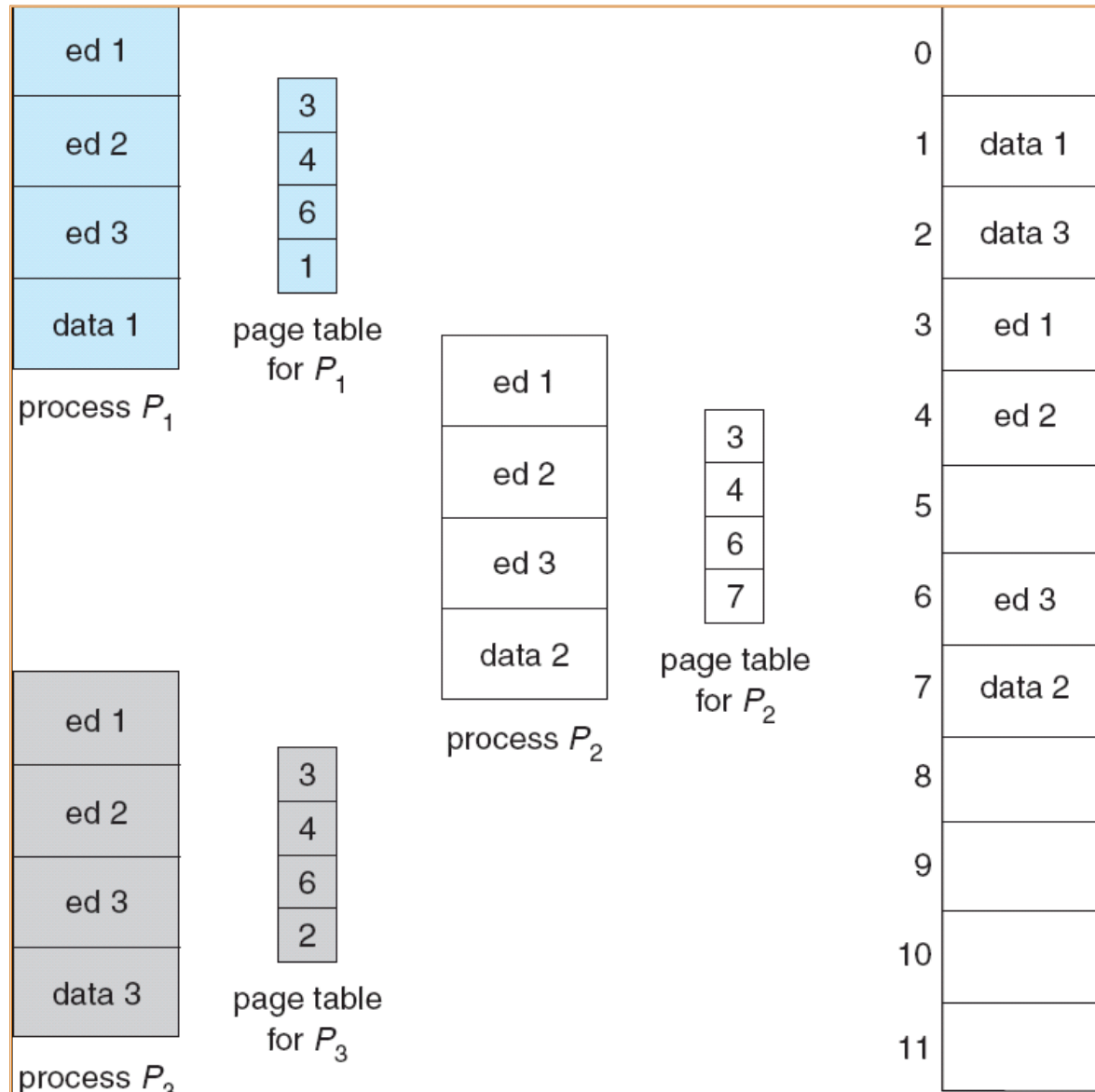
❖ Shared code

- **One copy of read-only** (reentrant) **code** shared among processes (i.e., text editors, compilers, window systems).
- **Shared code** must appear in **same location** in the **logical address space** of all **processes**

❖ Private code and data

- **Each process keeps a separate copy** of the code and data
- The **pages** for the **private code and data** can appear **anywhere** in the logical address space

Shared Pages Example



Practice

Consider a system with a frame size of 4 KB, if a process P has a logical address space of size 200200 bytes.

a) How many pages are needed to represent this process?

$$\left\lceil \frac{200200}{4096} \right\rceil = \lceil 48.876953125 \rceil = 49 \text{ frames numbered 0 to 48.}$$

b) What is the amount of internal fragmentation, if any?

$$4096 - (200200 \bmod 4096) = 4096 - 3592 = 504 \text{ bytes.}$$

c) Assume that page k of this process is assigned to frame k+1000. Find the virtual then the physical addresses for each of the following logical addresses of process P:

i. 66690:

$$\text{page number} = \left\lfloor \frac{66690}{4096} \right\rfloor = 16 \quad \text{and}$$

$$\text{the offset} = [66690 \bmod 4096] = 1154$$

The physical address in frame number and offset is (1016, 1154)

ii. 555011:

similar to i.

Practice

Consider a system with physical memory of 2 GB, a frame size of 4 KB, and the maximum size of logical address space supported by this system is 1 MB.

a) How many frames are there in this system?

$$\frac{2,147,483,648}{4096} = 524288 \text{ frames or } 2^{31}/2^{12} = 2^{19} = 524288 \text{ frames}$$

b) What is the maximum number of frames that can be allocated to a process in this system?

$$\frac{1MB}{4KB} = \frac{2^{20}}{2^{12}} = 2^8 = 256 \text{ frames}$$

c) How many bits are needed to represent the following:

i. The page number

8

ii. The offset

12

Practice

In a system with physical memory of 4 GB with a frame size of 8 KB, consider a process P with a logical address space of size 2 MB.

- a) If a page table entry consists of a frame number and the offset, how many bits is the size of the page table for this process?

Number of pages is $\left\lceil \frac{2MB}{8KB} \right\rceil = \left\lceil \frac{2^{21}}{2^{13}} \right\rceil = 2^8 = 256$ therefore there are 256 entries in the page table.

The number of frames in the system is $\frac{4GB}{8KB} = \frac{2^{32}}{2^{13}} = 2^{19}$ therefore, 19 bits to represent a frame number.

The page size is $8KB = 2^{13}$ therefore, 13 bits to represent the offset.

Hence, the size of the page table for process P is $256 \times (19 + 13) = 256 \times 32 = 8192 \text{bits} = 8KB$.

- b) What is the amount of internal fragmentation, if any?

Zero.

- c) If page p of this process is always located in frame p+1000, what is the logical address of the physical address **8315456**?

The frame number is $\left\lfloor \frac{8315456}{8192} \right\rfloor = \lfloor 1015.0703125 \rfloor = 1015 \rightarrow 15$ is the page number.

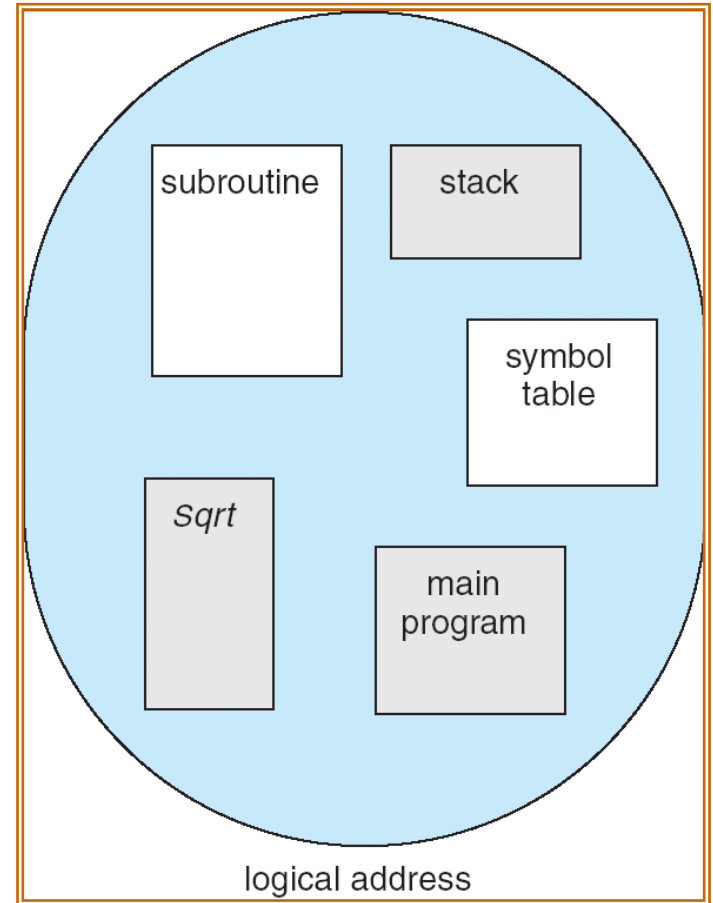
The offset is $(8315456 \bmod 8192) = 576$

Hence, (15 , 576) is the virtual address

and $15 \times 8192 + 576 = 123456$ is the logical address.

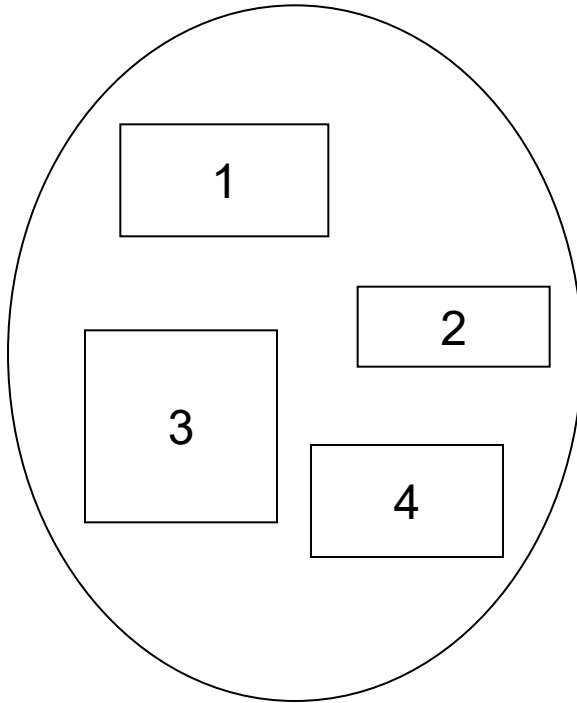
Segmentation

- ❖ Memory-management scheme that supports user view of memory
- ❖ A program is a collection of **segments**. A **segment** is a logical unit such as:
 - main program,
 - procedure,
 - function,
 - method,
 - object,
 - local variables, global variables,
 - common block,
 - stack,
 - symbol table, arrays

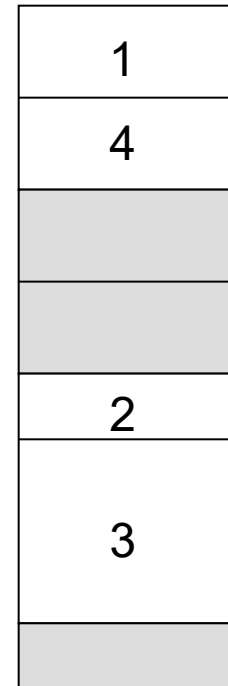


User's view of the program

Logical View of Segmentation



user space



physical memory space

Segmentation Architecture

❖ **Logical address** consists of a two tuple:

$\langle \text{segment-number}, \text{offset} \rangle$,

❖ **Segment table** – maps two-dimensional physical addresses; each table entry has:

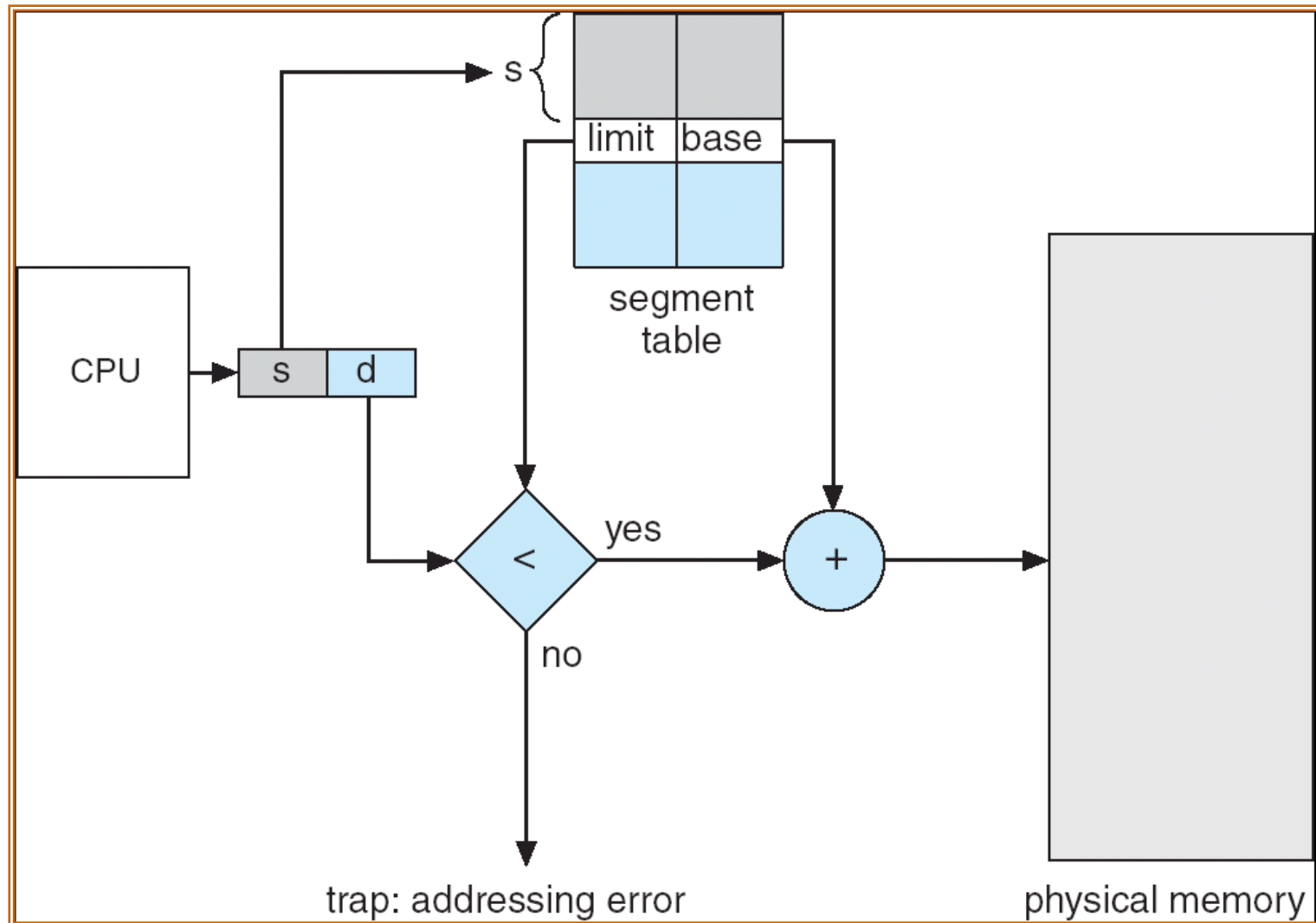
- **base** – contains the starting physical address where the segments reside in memory
- **limit** – specifies the length of the segment

❖ **Segment-table base register (STBR)** points to the segment table's location in memory

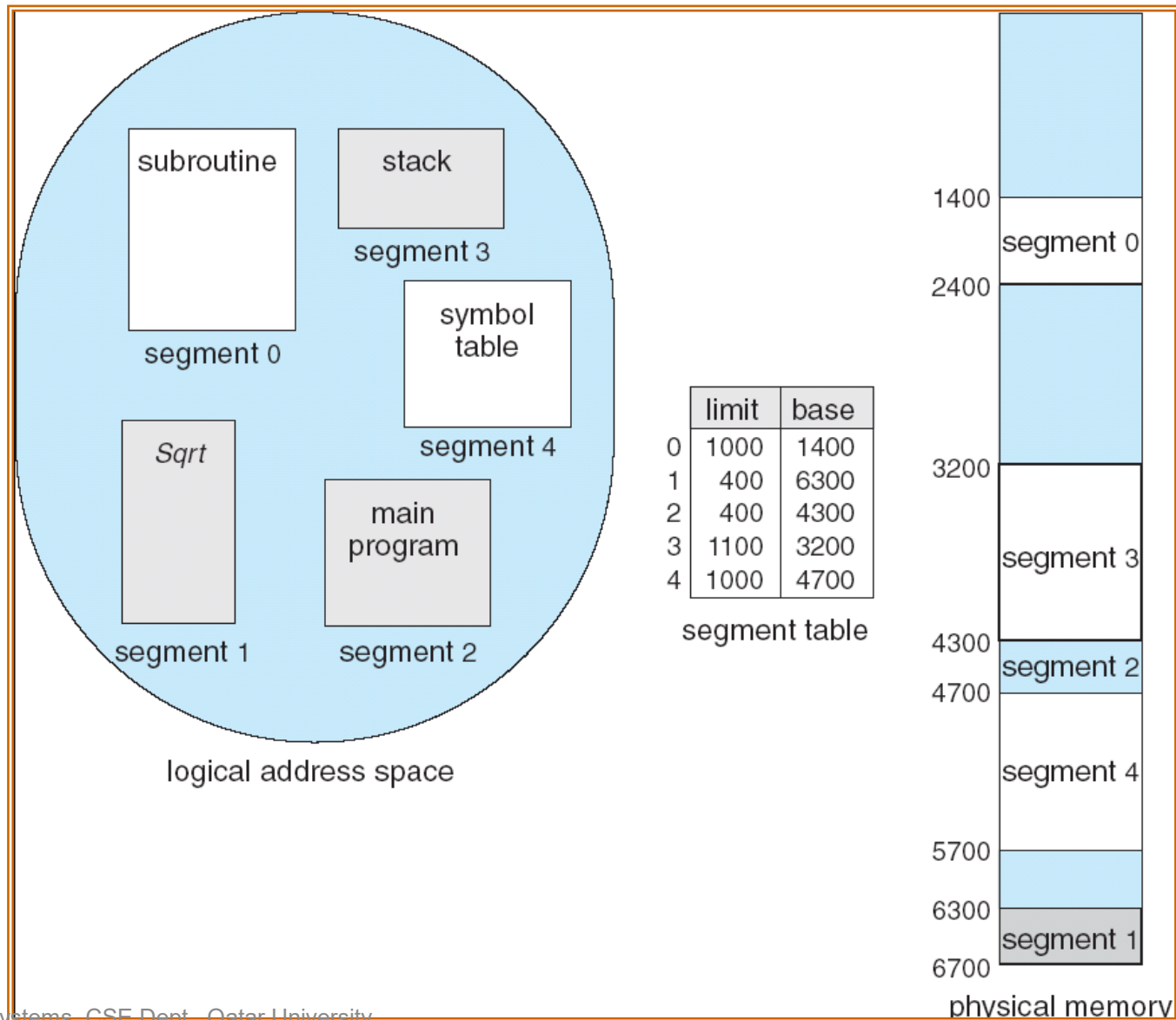
❖ **Segment-table length register (STLR)** indicates number of segments used by a program;

segment number s is legal if $s < \text{STLR}$

Segmentation Hardware



Example of Segmentation



Practice

Consider the following **segmentation table** for the **process P** that happen to have **zero internal fragmentation**.

Segment	Base	limit
0	11110000	5500
1	11140000	2600
2	10010000	1400
3	16170000	3800

- a) Find the physical address of virtual logical address (1, 2300).
11142300
- b) What is the logical address of the physical address 16171234?
(3, 1234)
- c) Find the physical address of the logical address 8300.
 $(8300 - 5500 - 2600) = 200$ (in segment 2)
physical address = 10010200

Practice

Consider the following **segmentation table** for the **process P** that happen to have **zero internal fragmentation**.

Segment	Base	limit
0	11110000	8600
1	11140000	3000
2	10010000	2400
3	16170000	500

a) What is the total size of the logical address space?

$$8600 + 3000 + 2400 + 500 = 14500$$

b) Find the physical address of virtual logical address of (2, 450).

$$10010450$$

c) Find the virtual logical address (segment, offset) of the logical address 11300.

$$11300 - 8600 = 2700 (< 3000 \text{ means in segment 1})$$

$$\text{Virtual logical address} = (1, 2700)$$